

Rapport de Projet

Simulation de Boids (C++ / SFML)

Nom Prénom: **BENFDAL EL Mahdi**

Nom Prénom: **ECH-CHAIRI Yassine**

Formation : **M1 LBD**

Module : COO Conception Orientée Objet

7 janvier 2026



Table des matières

1	Introduction	3
1.1	Objectifs principaux	3
2	Conformité au cahier des charges (boids.pdf)	3
2.1	Paramètres par défaut	3
2.2	Règles locales	3
2.3	Contraintes physiques	3
2.4	Interface utilisateur	3
3	Architecture générale du projet	4
4	Description détaillée des classes	4
4.1	Vec2<T> : vecteur 2D générique	4
4.2	DynamicArray<T> : structure de données maison	4
4.3	Boid : un agent de la simulation	4
4.4	Flock : gestion du groupe	4
4.5	Rule (abstraite) et règles dérivées	4
4.6	Settings : paramètres de la simulation	4
4.7	InterfaceMenuSFML et Renderer	4
4.8	Simulation	4
4.9	Sauvegarde et Chargement (SaveManager)	5
4.9.1	Structure du projet	5
5	Évolution du Diagramme UML	5
5.1	Comparaison UML initial vs UML final	5
6	Détails techniques d'implémentation	7
6.1	Gestion des bords	7
7	Extensions réalisées	8
7.1	Obstacle	8
7.2	Prédateur	8
8	Organisation technique et procédures de validation	8
8.1	Documentation et guides d'utilisation	8
8.2	Système de construction unifié (Makefile)	8
8.3	Benchmarks	8
8.4	Analyse de performance (Google Benchmark)	9
8.5	Périmètre des tests unitaires	9
9	Conclusion	9

1 Introduction

Ce projet consiste à implémenter une simulation de *Boids* (Reynolds), où chaque agent (boid) se déplace en suivant des règles locales simples. L'objectif est de faire émerger un comportement collectif réaliste (vol en groupe, cohésion, évitement des collisions), avec visualisation en temps réel via SFML, et une conception orientée objet rigoureuse (classes, polymorphisme, templates).

1.1 Objectifs principaux

- Implémenter les 3 règles fondamentales : **cohésion**, **séparation**, **alignement**.
- Respecter les contraintes de simulation : vitesse maximale, accélération maximale, mise à jour position/vitesse.
- Fournir une architecture **orientée objet** et extensible (polymorphisme des règles).
- Implémenter une structure de données maison : **DynamicArray<T>** (templates) pour remplacer la STL.
- Fournir une interface utilisateur et une visualisation (SFML).
- Ajouter des fonctionnalités complémentaires (extensions) : obstacle, prédateur, sauvegarde/-chargement.

2 Conformité au cahier des charges (boids.pdf)

Cette section décrit la conformité du projet vis-à-vis des exigences.

2.1 Paramètres par défaut

Les paramètres initialisés dans la classe **Settings** respectent les valeurs attendues (nombre de boids, dimensions, vitesse max, accélération max, rayons, poids des règles). Cela permet de reproduire directement le comportement demandé sans configuration supplémentaire.

2.2 Règles locales

Les trois règles sont implémentées sous forme de classes distinctes (une classe par règle), chacune calculant une force appliquée au boid courant en fonction de ses voisins :

- **Cohésion** : tendance à rejoindre le centre de masse des voisins.
- **Séparation** : force de répulsion si des voisins sont trop proches.
- **Alignement** : tendance à aligner la direction/vitesse sur les voisins.

Chaque règle produit une force, puis la simulation combine ces forces en appliquant des coefficients (w_{coh} , w_{sep} , w_{ali}).

2.3 Contraintes physiques

La mise à jour d'un boid respecte :

- une **accélération maximale** (force totale limitée),
- une **vitesse maximale** (norme de la vitesse limitée),
- une **mise à jour de la position** à partir de la vitesse et du temps écoulé (dt).

2.4 Interface utilisateur

Le projet propose :

- un **menu** SFML complet (écran principal, paramètres, chargement),
- des interactions clavier/souris pendant la simulation (reset, toggle rebond, obstacle, prédateur, sauvegarde).

3 Architecture générale du projet

L'architecture est organisée autour de trois responsabilités (MVC adapté) :

- **Modèle** (données et logique) : Boid, Flock, Rules, Settings, Vec2, DynamicArray.
- **Simulation** (orchestration) : boucle principale, gestion du temps, événements, application des règles.
- **Vue** (rendu/UX) : Renderer, InterfaceMenuSFML.

4 Description détaillée des classes

4.1 Vec2<T> : vecteur 2D générique

Vec2<T> représente un vecteur 2D utilisé pour les positions, vitesses et forces. Le choix d'un template permet d'utiliser différents types numériques si nécessaire. **Responsabilités** : opérations vectorielles (addition, soustraction, multiplication), calculs de norme et normalisation.

4.2 DynamicArray<T> : structure de données maison

DynamicArray<T> remplace les conteneurs STL (comme `std::vector`) qui étaient interdits pour le stockage des entités. **Responsabilités** : gestion manuelle de la mémoire (allocation dynamique), redimensionnement automatique, accès indexé.

4.3 Boid : un agent de la simulation

Un boid possède une position et une vitesse. Il est responsable de l'intégration de son mouvement (Application des forces → Accélération → Vitesse → Position).

4.4 Flock : gestion du groupe

Flock encapsule l'ensemble des boids. Il fournit l'initialisation aléatoire et la méthode `updateAll` qui orchestre le calcul des forces pour chaque agent.

4.5 Rule (abstraite) et règles dérivées

Rule est une classe abstraite pure. Le polymorphisme permet à la classe **Flock** de traiter une liste générique de règles sans connaître leur type concret (Cohésion, Obstacle, etc.), respectant ainsi le principe Ouvert/Fermé (Open/Closed Principle).

4.6 Settings : paramètres de la simulation

Singleton de fait (passé par référence), cette structure centralise tous les paramètres modifiables (physique, poids, dimensions) et leurs bornes de sécurité.

4.7 InterfaceMenuSFML et Renderer

Classes dédiées à l'affichage via SFML. **Renderer** gère le dessin géométrique des boids, tandis que **InterfaceMenuSFML** gère la navigation et les entrées utilisateur dans les menus.

4.8 Simulation

Classe maîtresse qui contient la boucle de jeu (*Game Loop*), gère les événements SFML et appelle les mises à jour du modèle et du rendu.

4.9 Sauvegarde et Chargement (SaveManager)

L'ajout de la classe **SaveManager** répond au besoin de persistance des données. Elle s'intègre au diagramme UML comme une dépendance utilisée par **Simulation** et **InterfaceMenuSFML**. **Fonctionnement :**

- **Sauvegarde :** Lors de l'appui sur une touche dédiée, l'état actuel des **Settings** (poids, nombre de boids, etc.) est sérialisé dans un fichier texte.
- **Chargement :** L'interface permet de sélectionner un fichier de configuration existant. Le **SaveManager** lit ce fichier et met à jour l'instance unique de **Settings**, permettant de reprendre une simulation avec des paramètres précis.

4.9.1 Structure du projet

La structure des fichiers est organisée de manière classique afin de séparer clairement les sources, headers, ressources et tests :

```
.
Makefile           # Script de build
Doxyfile           # Config Doxygen
README.md          # Documentation rapide
bin/               # Exécutables
assets/            # Ressources (Font, Saves)
include/           # Fichiers .h (Headers)
    bd/            # Namespace bd
src/               # Fichiers .cpp (Sources)
tests/            # Tests unitaires
```

5 Évolution du Diagramme UML

5.1 Comparaison UML initial vs UML final

Au début du projet, nous avons conçu un **diagramme UML initial** (Image 1) afin de définir l'architecture de base et répartir le travail. Une fois le projet terminé, nous avons constaté que les fonctionnalités ajoutées ont conduit à un **UML final** (Image 2) plus complet.

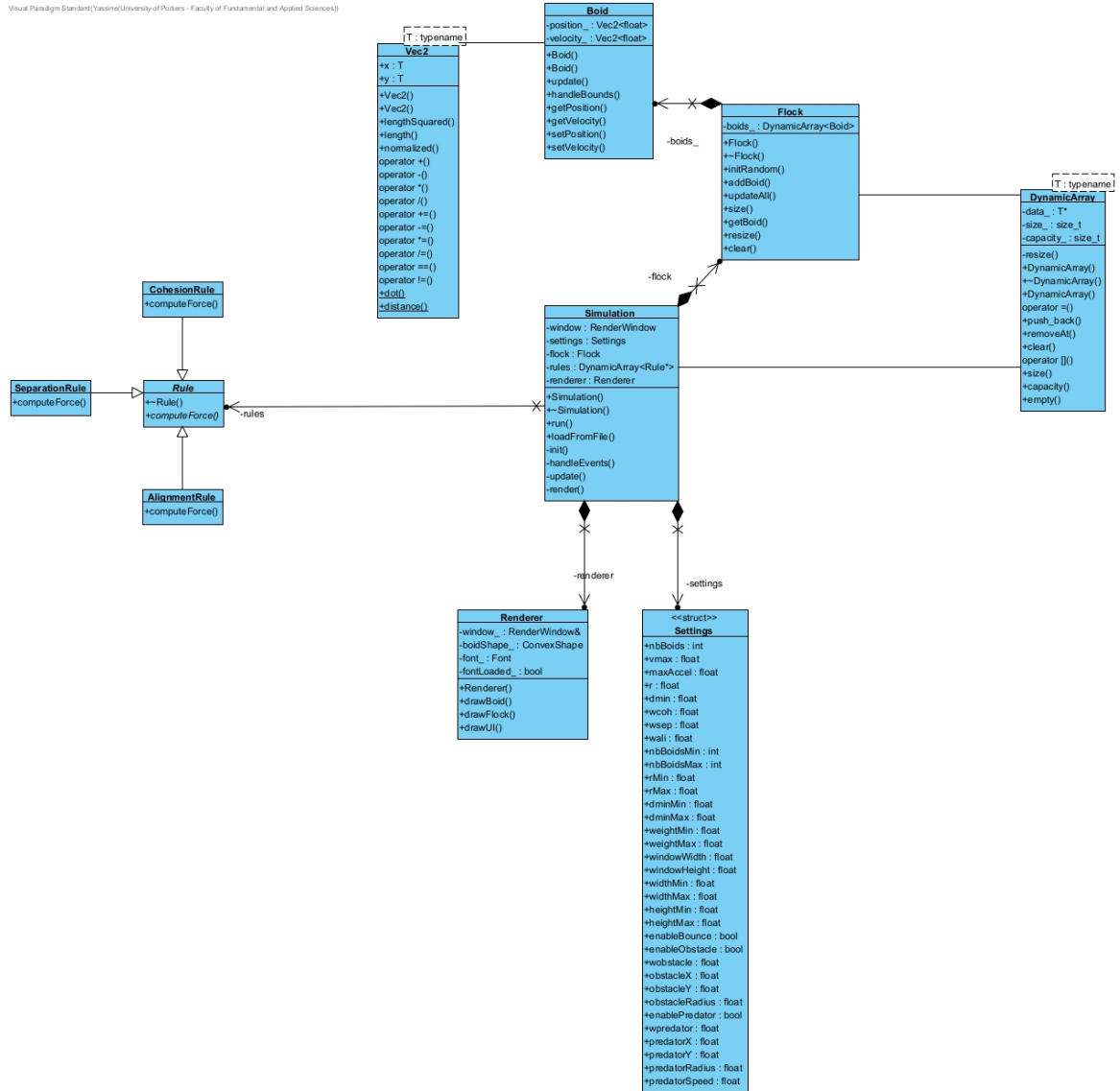


FIGURE 1 – UML initial (début du projet)

Visual Paradigm Standard (Yassine/University of Poitiers - Faculty of Fundamental and Applied Sciences)

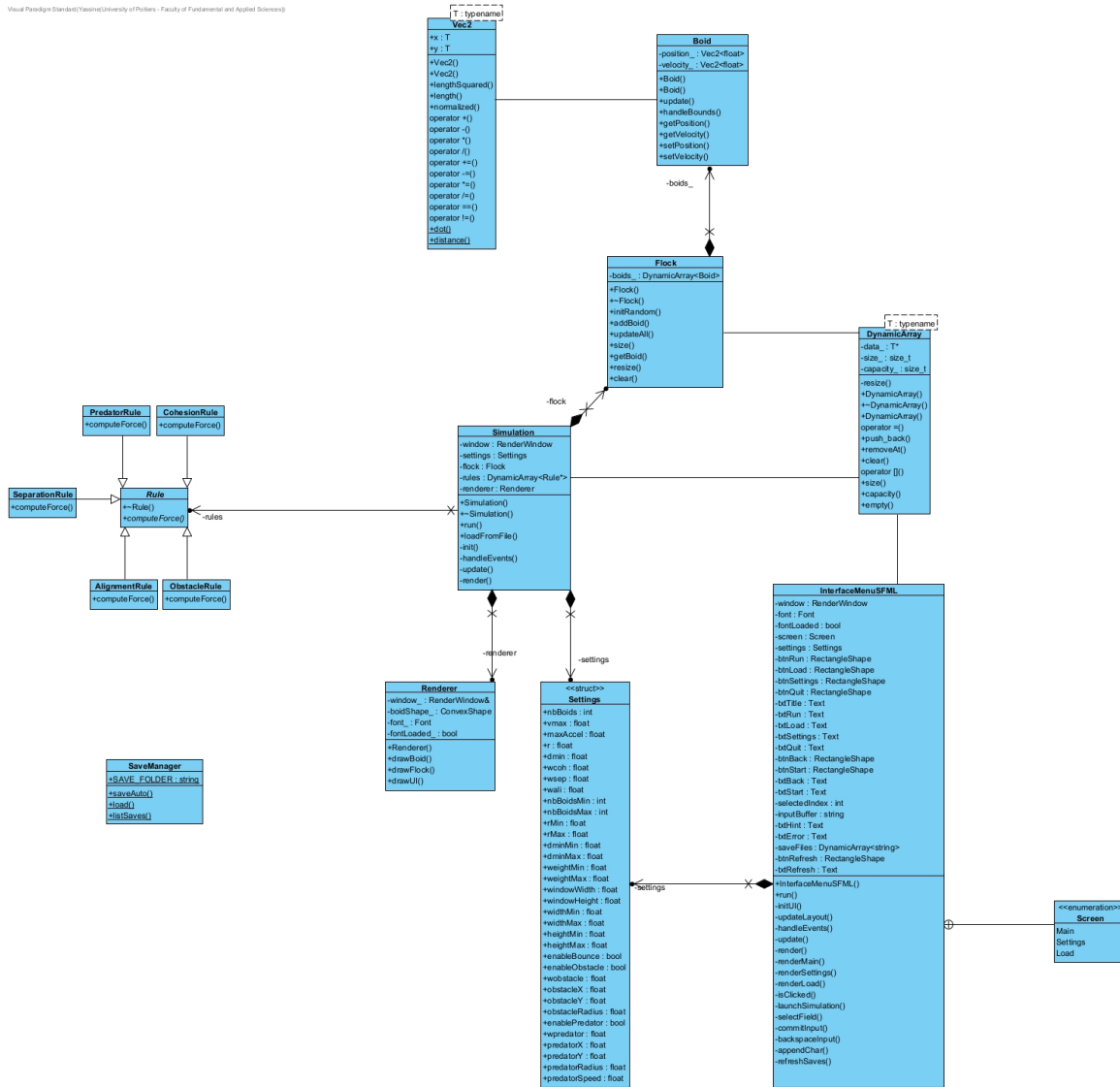


FIGURE 2 – UML final (fin du projet)

Les changements majeurs constatés entre la conception initiale et la version finale sont :

- **Ajout de nouvelles règles** : Extension du polymorphisme vers `ObstacleRule` et `PredatorRule`.
- **Interface Menu** : Ajout de la classe `InterfaceMenuSFML` pour gérer la navigation avant la simulation.
- **Persistance** : Ajout de la classe utilitaire `SaveManager` pour gérer les flux de fichiers, ce qui n'était pas prévu dans le diagramme initial.

6 Détails techniques d'implémentation

Cette section détaille des choix techniques spécifiques implémentés dans le code fourni.

6.1 Gestion des bords

Afin de maintenir les boids dans la zone visible, deux modes de gestion des limites ont été implémentés dans la méthode `Boid::handleBounds` :

- **Mode Wrap**) : C'est le comportement par défaut. Lorsqu'un boid sort d'un côté de la fenêtre (ex : droite), il réapparaît instantanément de l'autre côté (ex : gauche). Cela simule un espace infini et continu.

- **Mode Bounce (Rebond)** : Activable via les paramètres (`enableBounce`). Lorsqu'un boid atteint une marge définie près du bord, sa vitesse sur l'axe concerné est inversée, simulant un rebond physique contre une paroi.

7 Extensions réalisées

7.1 Obstacle

Ajout d'un obstacle manipulable (placement à la souris). Les boids appliquent une force d'évitement lorsqu'ils se rapprochent trop de l'obstacle, utilisant une force de "steering" opposée.

7.2 Prédateur

Ajout d'un prédateur activable au clavier. Les boids fuient le prédateur (force de répulsion/panique), ce qui modifie le comportement collectif de manière dynamique.

8 Organisation technique et procédures de validation

L'architecture du projet a été conçue pour séparer clairement le développement de l'application principale de sa phase de validation et d'analyse de performance. Cette organisation se reflète dans la structure des fichiers de documentation, le système de construction (Build System) et l'isolation des benchmarks.

8.1 Documentation et guides d'utilisation

Afin de faciliter la prise en main et la maintenance du projet, la documentation technique a été scindée en deux fichiers distincts, chacun répondant à un besoin spécifique :

- **README.md (Racine)** : Ce document constitue le point d'entrée principal du projet. Il détaille l'installation des dépendances (SFML, compilateur C++), les instructions de compilation de l'application graphique, ainsi que le manuel utilisateur pour interagir avec la simulation (commandes clavier, gestion des sauvegardes).
- **Bench/README.md (Dossier Bench)** : Ce document est spécifiquement dédié au module d'analyse de performance. Il explicite la méthodologie de test utilisée, le fonctionnement de la bibliothèque *Google Benchmark*, et fournit les instructions pour interpréter les résultats de complexité algorithmique ($O(N^2)$).

8.2 Système de construction unifié (Makefile)

Le projet repose sur un **Makefile** unique et modulaire, situé à la racine. Ce choix garantit une cohérence globale dans le processus de compilation et simplifie l'expérience développeur. Le Makefile orchestre trois cibles principales.

8.3 Benchmarks

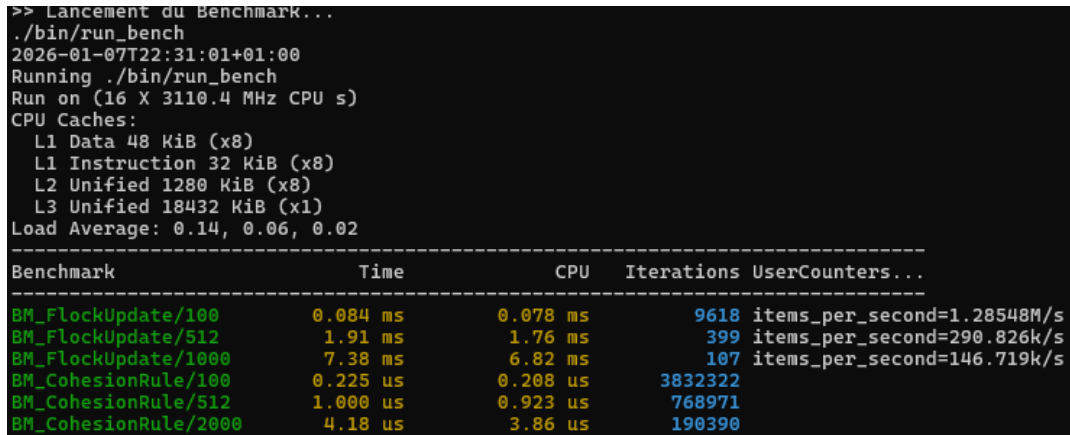
Une attention particulière a été portée à la séparation entre le code applicatif et les outils de mesure de performance. Les benchmarks (utilisant *Google Benchmark*) sont isolés dans un répertoire dédié (**Bench/**) et compilés séparément. Cette architecture répond à deux objectifs majeurs :

- **Intégrité des performances** : Elle garantit que le code de mesure n'interfère pas avec l'exécutable principal. Le programme de simulation reste ainsi léger et exempt de toute surcharge liée aux bibliothèques de profiling.
- **Distinction des préoccupations** : Elle sépare clairement la validation fonctionnelle (Tests Unitaires : "Le code fonctionne-t-il correctement?") de l'analyse de performance (Benchmarks : "Le code est-il rapide?"). Cela permet d'exécuter des scénarios de test intensifs (milliers d'agents) sans impacter la stabilité de l'application graphique.

8.4 Analyse de performance (Google Benchmark)

Dans une démarche d’approfondissement technique, nous avons intégré la bibliothèque **Google Benchmark** afin de mesurer précisément la complexité de notre moteur, en isolant le temps de calcul CPU du rendu graphique.

Contrairement à une mesure manuelle, cet outil garantit la fiabilité des résultats (gestion du *warm-up*, répétition statistique, compilation optimisée `-O3`).



```

>> Lancement du Benchmark...
./bin/run_bench
2026-01-07T22:31:01+01:00
Running ./bin/run_bench
Run on (16 X 3110.4 MHz CPU s)
CPU Caches:
  L1 Data 48 KiB (x8)
  L1 Instruction 32 KiB (x8)
  L2 Unified 1280 KiB (x8)
  L3 Unified 18432 KiB (x1)
Load Average: 0.14, 0.06, 0.02
-----
Benchmark                                Time          CPU    Iterations  UserCounters...
-----
BM_FlockUpdate/100                       0.084 ms      0.078 ms    9618 items_per_second=1.28548M/s
BM_FlockUpdate/512                       1.91 ms       1.76 ms     399 items_per_second=290.826k/s
BM_FlockUpdate/1000                      7.38 ms       6.82 ms     107 items_per_second=146.719k/s
BM_CohesionRule/100                      0.225 us      0.208 us   3832322
BM_CohesionRule/512                      1.000 us      0.923 us   768971
BM_CohesionRule/2000                     4.18 us       3.86 us   190390

```

FIGURE 3 – Temps de calcul par frame en fonction du nombre de Boids

Résultats et interprétation : L’analyse des mesures (Fig. 3) confirme expérimentalement la complexité quadratique ($O(N^2)$) de l’algorithme naïf :

- Pour $N = 512$ boids, le temps de calcul est de **1.82 ms**.
- Pour $N = 1000$ boids (doublement de la population), le temps passe à **7.23 ms** (multiplication par ≈ 4).

Ces métriques démontrent que notre moteur peut gérer de manière fluide (60 FPS, soit un budget de 16 ms) jusqu’à environ **1500 agents** simultanés avant saturation du CPU.

8.5 Périmètre des tests unitaires

Outre l’architecture, le projet contient des tests unitaires concrets pour vérifier la fiabilité des briques de base :

- les opérations de **Vec2** (norme, normalisation, opérateurs),
- le bon fonctionnement de **DynamicArray** (redimensionnement, insertion, suppression),
- la cohérence des règles physiques (forces cohésion/séparation/alignement).

Ces tests assurent une base fiable et ont facilité le développement itératif.

9 Conclusion

Le projet met en place une simulation Boids fonctionnelle, respectant scrupuleusement les principes de la Conception Orientée Objet. L’utilisation du polymorphisme pour les règles permet une grande extensibilité (ajout aisé de nouvelles règles), tandis que l’implémentation manuelle des structures de données (**DynamicArray**) démontre une maîtrise de la gestion mémoire en C++. L’ajout d’une interface graphique complète et de fonctionnalités de sauvegarde transforme la simulation théorique en une application interactive utilisable. Le cahier des charges a été respecté, tant sur les aspects algorithmiques que sur les contraintes techniques imposées.

L’intégralité du code source du projet ainsi que l’historique de développement sont disponibles en libre accès sur le dépôt GitHub suivant :

https://github.com/YassineECh-23/Project_Boids